DocIngestQA

DocIngestQA / Interview Defense Document

Sidharth Kriplani

Document v1.0 · Sourced exclusively from repo files

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact or DB row
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

DocIngestQA Interview Defense Document v3.0

Complete Edition — Pre-Indexing RAG Document Ingestion Auditing

SECTION 1 — PROJECT IDENTITY

1.1 One-Sentence Summary

DocIngestQA audits exported document chunks before they are indexed into a vector store by running 11 deterministic checks — covering metadata completeness, document coverage, page coverage, chunk length, OCR noise, duplicate detection, source distribution, chunk overlap, encoding health, and split quality — and produces a structured PASS/WARN/FAIL report that gates indexing.

1.2 The Problem This Solves

RAG pipelines fail silently at ingestion. Missing pages, OCR garble, duplicate chunks, and mojibake encoding errors all make it into the vector database undetected — then show up as hallucinations or missed retrievals in production. By the time you trace the error back to a bad chunk, you have already shipped the issue. DocIngestQA moves that quality gate to before indexing, where it is cheap to fix.

The fundamental issue is that vector databases accept any text you give them. They don't know that a chunk is a repeated fragment, a garbled OCR output, or an encoding-corrupted paragraph. The index looks fine. The embedding looks fine. The retrieval result looks fine. The problem only manifests when the model hallucinates based on a corrupted context or fails to retrieve because the relevant chunk was OCR-garbled into unrecognizable tokens.

1.3 Interpretation Note

DocIngestQA checks ingestion quality only. It does not assess retrieval relevance, answer faithfulness, semantic coherence, or chunk informativeness. A corpus that passes all 11 checks is free from structural defects — it is not guaranteed to produce good retrieval results. This note is mandatory on every output and cannot be suppressed.

SECTION 2 — THE 11 CHECKS

2.1 v0.1 Checks (8 checks)

Check 1: Input Summary — Validates the basic structure of the input JSONL: required fields (`chunk_id`, `source`, `page`, `text`) are present in every row, no null chunk IDs, no empty text fields. A missing required field produces FAIL; null IDs or empty text produce WARN.

Check 2: Metadata Completeness — Beyond required fields, optional metadata fields (`section`, `doc_type`, `language`, `timestamp`) are checked for completeness. Chunks with missing optional metadata are flagged WARN at the field level, with a count of affected chunks. Incomplete metadata reduces the quality of filtered retrieval (e.g., filtering by doc_type or language at query time).

Check 3: Document Coverage — If a `source_manifest.json` is provided listing expected source documents, this check verifies that every expected document has at least one chunk in the corpus. Missing documents produce FAIL — a document that was expected but not chunked represents a coverage gap that will produce retrieval blind spots. No manifest = INSUFFICIENT_INPUT.

Check 4: Page Coverage — For each document in the manifest, the check verifies that page numbers in the chunk corpus form a contiguous sequence with no large gaps. A 50-page document with chunks from pages 1–20 and 40–50 but nothing for pages 21–39 indicates a chunker failure or a corrupted extraction. Page gaps above `max_page_gap` (default: 3) produce WARN with the gap range identified.

Check 5: Chunk Length — Chunks below `min_chunk_tokens` (default: 10) or above `max_chunk_tokens` (default: 512) are flagged WARN. Too-short chunks (single sentences, headers) carry insufficient context for retrieval. Too-long chunks exceed most embedding model context windows and must be truncated, potentially losing the most relevant text.

Check 6: OCR Noise — Each chunk is scanned for OCR corruption indicators: high non-alphabetic character density (`non_alpha_ratio ≥ 0.25`), runs of repeated characters (e.g., "aaaaaaaa" from a

misread image), and the presence of common OCR error tokens (`"l1l1"`, `"0O0O"`, garbled ligatures). Affected chunks are flagged WARN with the noise ratio and sample text.

Check 7: Duplicate Chunks — Exact content duplicates are detected using SHA-256 hashes of normalised chunk text (lowercase, collapsed whitespace). Near-exact duplicates (differing only in whitespace or punctuation) are grouped by hash. Duplicate chunks waste index capacity and can amplify irrelevant content in retrieval by artificially inflating the apparent relevance of a repeated passage.

Check 8: Source Distribution — Checks for extreme imbalance in the number of chunks per source. If any single source contributes more than `max_source_fraction` (default: 50%) of all chunks, it is flagged WARN — the corpus is dominated by one document and retrieval will be biased toward it regardless of query relevance.

2.2 v0.2 Checks (3 checks)

Check 9: Chunk Overlap — For consecutive chunks from the same source and page sequence, the check computes the token overlap ratio at chunk boundaries. Overlap > `max_overlap_ratio` (default: 0.3) between adjacent chunks means 30%+ of each chunk is repeated in the next — wasteful for retrieval and potentially causing the same passage to appear multiple times in the context window. Zero overlap is also flagged WARN (hard boundary splits can cut sentences mid-thought).

Check 10: Encoding Health — Each chunk's text is scanned for encoding artifacts: mojibake sequences (UTF-8 decoded as Latin-1 and re-encoded, producing sequences like `â€™` for apostrophe), null bytes, and control characters. Encoding issues corrupt semantic meaning and produce nonsensical embeddings. This check produces FAIL if more than `encoding_fail_threshold` (default: 5%) of chunks are affected.

Check 11: Split Quality — Checks whether chunks end at natural sentence boundaries. A chunk that ends mid-sentence (no terminal punctuation, and the following word is lowercase) has been split poorly — the chunk's last sentence is incomplete, reducing its retrieval usefulness. Chunks failing this check produce WARN with the truncated ending shown.

SECTION 3 — SEVERITY LEVELS

3.1 Status Hierarchy

DocIngestQA uses a four-level status hierarchy: FAIL > WARN > INSUFFICIENT_INPUT > PASS.

FAIL: Structural issues that will definitively degrade retrieval: missing required fields (Check 1), missing expected documents (Check 3), encoding corruption above threshold (Check 10). These must be resolved before indexing.

WARN: Quality issues that reduce retrieval quality but don't break the index: page gaps, short/long chunks, OCR noise, duplicates, source imbalance, split quality. These should be reviewed and addressed where feasible.

INSUFFICIENT_INPUT: Checks that cannot run without optional configuration: document coverage (no manifest), near-duplicate detection if corpus is empty. Documented explicitly — INSUFFICIENT_INPUT is not PASS.

PASS: No issues detected by this check.

The overall report status is the maximum severity across all checks.

SECTION 4 — API AND USAGE

4.1 Core API

```
from docingestqa import AuditConfig, IngestionAuditor auditor = IngestionAuditor(
chunks_path="chunks.jsonl", documents_path="source_manifest.json", # optional config=AuditConfig(
min_chunk_tokens=10, max_chunk_tokens=512, max_source_fraction=0.50, max_overlap_ratio=0.30,
encoding_fail_threshold=0.05, ), ) report = auditor.run() print(report.overall_status) # FAIL / WARN
/ INSUFFICIENT_INPUT / PASS print(report.to_markdown()) report.to_json("audit.json")
report.to_html("audit.html")
```

4.2 CLI Usage

```
# Audit chunks and write report python -m docingestqa audit chunks.jsonl --manifest
source_manifest.json --output audit.json # Gate a CI pipeline python -m docingestqa audit
chunks.jsonl && echo "Audit passed" || echo "Audit failed"
```

Exit codes: PASS = 0, WARN = 1, FAIL = 2, INSUFFICIENT_INPUT = 3. Directly usable in shell scripts and CI pipelines.

4.3 Source Manifest Format

```
[ {"source": "user_guide.pdf", "expected_pages": 45}, {"source": "api_reference.pdf",
"expected_pages": 102}, {"source": "changelog.pdf", "expected_pages": 18} ]
```

The manifest enables document coverage and page coverage checks. Without it, those two checks return INSUFFICIENT_INPUT.

SECTION 5 — WHERE THIS FITS IN A RAG PIPELINE

5.1 Pre-Indexing Gate

DocIngestQA runs at the boundary between document extraction and index construction:

```
Raw Documents (PDF, HTML, DOCX) ■ ▼ Chunking Pipeline (LangChain, LlamaIndex, custom) ■ ▼ Export
chunks.jsonl ■ ▼ DocIngestQA Audit ← HERE ■ FAIL → Stop, fix extraction WARN → Review, optionally
proceed PASS → Proceed to indexing ■ ▼ Vector Store Indexing (Chroma, Pinecone, Weaviate) ■ ▼ RAG
Application
```

5.2 Relationship to GoldenSetAuditor

DocIngestQA and GoldenSetAuditor are complementary quality gates for different parts of the RAG pipeline:

- **DocIngestQA** audits the source corpus before indexing — are the chunks structurally sound?
- **GoldenSetAuditor** audits the evaluation dataset before benchmarking — are the reference answers trustworthy?

Both must pass for a RAG system's evaluation metrics to be meaningful. A system with clean chunks but conflicting golden set labels produces unreliable benchmark scores. A system with clean golden labels but

OCR-corrupted chunks produces low retrieval metrics that reflect chunk quality, not model quality.

SECTION 6 — DESIGN DECISIONS

6.1 Why Deterministic Checks Only

All 11 checks are deterministic: given the same input, they always produce the same output. No LLM calls, no embedding similarity, no model inference. This is deliberate: a quality gate that requires LLM calls to evaluate is (1) expensive to run on every ingestion, (2) non-reproducible (LLM outputs vary), and (3) circular (using an LLM to evaluate chunks that will be used for LLM retrieval). Deterministic checks are cheap, reproducible, and unambiguous.

6.2 Why PyPI-Published

DocIngestQA is published to PyPI (`pip install docingestqa`) so it can be installed in any RAG team's environment without cloning the repository. The CLI interface makes it usable in CI pipelines as a pre-indexing step without writing Python code. The single-package install with zero external dependencies (beyond the standard library) keeps it lightweight.

6.3 Why HTML Output

The HTML output format is a self-contained single-file report that can be shared with non-technical stakeholders: the HTML includes a color-coded findings table (FAIL in red, WARN in amber, PASS in green) with chunk-level evidence (source name, page number, text preview). This makes DocIngestQA reports reviewable by document owners who don't read JSON.

SECTION 7 — Q&A

Q: What's the most common FAIL finding in practice?

Encoding corruption (Check 10). PDF extraction is notoriously encoding-fragile: PDFs that use non-standard font encoding, CJK character sets, or ligature substitutions frequently produce mojibake when extracted with common libraries (pdfplumber, PyMuPDF). The pattern is usually systematic — a specific document type or extraction configuration that affects all chunks from those sources.

Q: Why is chunk overlap both too much and too little a problem?

Too much overlap (> 30%): wasteful index size; the same text appears in multiple chunks; retrieval returns near-duplicate passages as top results, consuming the model's context window with repetition. Too little overlap (zero, hard cuts): sentences are split mid-thought; the first chunk loses its concluding sentence and the second chunk starts with a dangling continuation. The optimal overlap (10-20%) ensures each chunk is self-contained while providing continuity for boundary sentences.

Q: How does DocIngestQA handle multi-language corpora?

The `language` field in chunk metadata is validated for completeness in Check 2. OCR noise detection (Check 6) uses character class patterns that are primarily Latin-alphabet based — for CJK corpora, the non-alpha ratio check is automatically disabled when the detected language is Chinese, Japanese, or Korean (the `language` metadata field drives this). Split quality detection (Check 11) uses sentence boundary patterns that are language-specific; non-English corpora should configure the `language` parameter to select the appropriate sentence boundary rules.

Q: What's the difference between DocIngestQA and a simple lint script?

A lint script checks syntax. DocIngestQA checks semantics: the chunks are syntactically valid JSON, but are they semantically useful for retrieval? Page coverage gaps, OCR noise, and split quality are semantic properties of the content that require understanding of what a good chunk looks like. DocIngestQA packages that semantic knowledge into systematic checks with configurable thresholds and structured evidence output.

SECTION 8 — NUMBERS AT A GLANCE

Metric	Value
Checks	11 (8 in v0.1, 3 in v0.2)
Checks that can FAIL	3 (input structure, doc coverage, encoding health)
Checks that WARN	8
Current version	v0.2.0
Output formats	JSON, Markdown, HTML
PyPI installable	Yes (`pip install docingestqa`)
CLI available	Yes (`python -m docingestqa audit`)

DocIngestQA Interview Defense Document v3.0 — Complete Edition

11 Checks · FAIL on Structural Defects · Pre-Indexing Gate · PyPI Published

SECTION 9 — DEMO WALKTHROUGH

9.1 Example Corpus and Findings

The demo corpus (`examples/assets/chunks.jsonl`) includes 150 synthetic chunks across 3 source documents with deliberately injected issues:

- 8 chunks with OCR noise (non-alpha ratio > 0.25)
- 4 exact duplicate chunk pairs
- 2 chunks below min_chunk_tokens (headers only)
- 1 document with a page 12–18 coverage gap
- 3 chunks with hard-cut split boundaries (mid-sentence endings)

Running the auditor on the demo corpus produces:

```
Overall Status: WARN Check Results: input_summary: PASS metadata_completeness: WARN (12 chunks missing 'section' field) document_coverage: WARN (1 of 3 documents has page gap 12-18) page_coverage: WARN (pages 12-18 missing in api_reference.pdf) chunk_length: WARN (2 chunks below min_chunk_tokens=10) ocr_noise: WARN (8 chunks with non_alpha_ratio >= 0.25) duplicate_chunks: WARN (4 exact duplicate pairs found) source_distribution: PASS chunk_overlap: WARN (3 chunks with hard-cut boundaries) encoding_health: PASS split_quality: WARN (3 chunks end mid-sentence)
```

9.2 Fixing the Issues

The demo also ships with `fix_demo_issues.py` which shows how to address each finding programmatically: re-run the PDF extractor on the page-gap document, deduplicate chunk content by hash, re-chunk the OCR-noisy sections with a higher resolution scan, and adjust the chunking boundary to sentence terminators. After fixing, the audit produces overall PASS.

This demo-to-fix workflow is the intended development loop: audit → identify findings → fix extraction config → re-audit.

DocIngestQA Interview Defense Document v3.0 — Complete Edition

11 Checks · FAIL on Structural Defects · Pre-Indexing Gate · PyPI Published · Demo Walkthrough